

Leakage-Controlled Concept Graph Construction and Cold-Start Diagnostic Protocol for Knowledge Tracing

Dao Minh Tuan ¹, Nguyen Tien Duong ¹,
Nguyen Khanh Trinh ¹, Ngo Quoc Khanh ³,
Nguyen Van Hau ^{1*}, Le Hoang Son ²

^{1*}Department of Quality Assurance and Testing, Hung Yen University
of Technology and Education, Hung Yen, Vietnam.

²Informatics, VNU University of Science, Vietnam National University
(VNU), Hanoi, Vietnam.

³Advanced International Research Center on Applied Artificial
Intelligence, Information Technology Institute, Vietnam National
University, Hanoi, Vietnam.

*Corresponding author(s). E-mail(s): [nvhou666@gmail.com](mailto:nvhau666@gmail.com);
Contributing authors: tuanymc@utehy.edu.vn; nduongnt@gmail.com;
trinhnkt@gmail.com; quockhanhngo.official@gmail.com;
sonlh@vnu.edu.vn;

Abstract

Graph-enhanced knowledge tracing (KT) often infers concept graphs from the same interaction logs used for training and evaluation. When edges are pooled before the train/test split, held-out information can reach the model through the graph even if the KT optimiser sees only the training fold. We present a leakage-controlled *audit protocol*—not a new KT backbone—for knowledge-component (KC) graph construction: learner-based temporal splits, train-only prerequisite inference (optional similarity where the Q-matrix supports it), DAG validation with cycle pruning, KC-level cold-start stratification, a practitioner checklist, and a DAG Disruption Rate (DDR) diagnostic. We instantiate the protocol on XES3G5M (primary model-comparison benchmark), ASSISTments 2012 (secondary), Junyi Academy (saturated sanity + ground-truth cross-validation), and two synthetic logs. The audit yields acyclic train-only prerequisite graphs on all corpora; the DDR sweep separates five augmentation operators, with a

prerequisite-preserving operator measurably less disruptive than edge dropping at a matched budget. An *exploratory* DGEKT-only downstream stress test on ASSISTments and XES3G5M finds near-zero AUC changes despite large structural perturbation; we do not extrapolate that null sensitivity to other backbones. On XES3G5M, leakage-controlled auditing changes deployable model ranking (e.g. GKT trails *simpleKT* by ≈ 0.041 AUC). Ground-truth cross-validation on Junyi shows node-level agreement but edge-level divergence between behavioural inference and expert curation. A deliberate full-log graph variant shifts headline metrics by at most **0.003** on the three public benchmarks studied here; we position leakage control as an applied reproducibility and decision-support precondition, not as a route to higher headline accuracy.

Keywords: Knowledge tracing, Concept graphs, Leakage control, DAG audit, Ground-truth cross-validation, Cold-start diagnostics, Reproducibility

1 Introduction

Knowledge tracing (KT) estimates a learner’s latent mastery of knowledge components from interaction history, supporting adaptive practice, intelligent tutoring, and curriculum recommendation. The field has progressed from Bayesian Knowledge Tracing [Corbett and Anderson \(1994\)](#) to deep sequential models [Piech et al. \(2015\)](#); [Zhang et al. \(2017\)](#); [Ghosh et al. \(2020\)](#); [Liu et al. \(2023a\)](#); [Pandey and Karypis \(2019\)](#); [Shen et al. \(2020\)](#) and graph-enhanced models that propagate information over relations between knowledge components [Nakagawa et al. \(2019\)](#); [Tong et al. \(2020\)](#); [Yang et al. \(2021\)](#); [Cheng et al. \(2024\)](#); [Cui et al. \(2024\)](#); [Tong et al. \(2022\)](#). These graph-based models rely on auxiliary concept graphs, usually encoding prerequisite or similarity relations.

Problem setting.

Let $\mathcal{D} = \{(u, t, q, c, y)\}$ be a KT interaction log with learner u , timestamp t , item q , tagged knowledge component c , and correctness label $y \in \{0, 1\}$. A learner-based temporal split $\mathcal{F} = (\mathcal{F}_{\text{train}}, \mathcal{F}_{\text{val}}, \mathcal{F}_{\text{test}})$ assigns each learner to exactly one fold. For evaluation fold f , the protocol must construct a fold-specific concept graph with prerequisite edges and, when the Q-matrix permits, optional similarity edges: $G_f = (\mathcal{C}, E_{\text{pre}}^f \cup E_{\text{sim}}^f)$ using *only* $\mathcal{F}_{\text{train}}^f$, then train a KT model on $\mathcal{F}_{\text{train}}^f$ and score predictions on $\mathcal{F}_{\text{test}}^f$. The central question is whether reported graph-augmented KT gains remain valid when G_f is audited for leakage, acyclicity, and cold-start coverage.

Figure 1 contrasts the two workflows. In workflow (a), prerequisite or similarity edges may be supported by validation/test interactions; the KT module then appears to generalise while part of the structural input already encodes future behaviour. Workflow (b) closes this channel by making graph construction a first-class, fold-conditioned step.

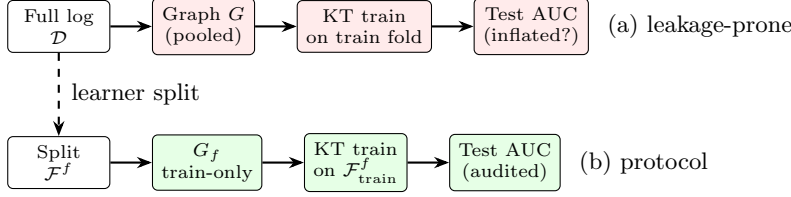


Fig. 1: Illustrative comparison of graph construction workflows. **(a)** Pooling the full log before edge inference lets held-out transitions influence G even when the KT optimiser sees only the train fold. **(b)** Our protocol builds G_f from $\mathcal{F}_{\text{train}}^f$ only and audits provenance before evaluation.

Real-world significance.

Leakage-controlled concept graphs matter wherever KT systems drive scheduling and gating decisions in ITS platforms, MOOC pipelines, and corporate training [Heffernan and Heffernan \(2014\)](#); [Junyi Academy \(2015\)](#); [Choi et al. \(2020\)](#); [Liu et al. \(2023b\)](#). Without fold-specific audits, reported AUC improvements are difficult to attribute to modelling versus preprocessing; our protocol makes graph provenance, DAG validity, and cold-start strata explicit reporting artefacts.

Fit for applied intelligence.

Applied Intelligence readers typically expect methods that improve the *trustworthy deployment* of intelligent systems, not only higher offline scores. Our contribution is an audit layer that reduces preventable mistakes in graph-enhanced KT pipelines: leaking graph evidence across splits, deploying acyclicity violations silently, reporting aggregate AUC on repetition-saturated corpora, or treating behavioural graphs as expert prerequisites. The deliverable is a reproducible checklist, provenance artefacts, and diagnostics that can change *which model a practitioner reports and deploys*, even when headline accuracy does not move.

Audit case study (XES3G5M).

On our primary benchmark, leakage-controlled evaluation separates graph backbones more clearly than on repetition-heavy corpora. Under train-only E_{pre} and three-fold learner CV, *simpleKT* reaches AUC ≈ 0.875 while GKT reaches only ≈ 0.834 (mean $\Delta \approx -0.041$ across folds); SKT and DyGKT collapse further (≈ 0.625 – 0.752). GIKT remains competitive (≈ 0.878). A deployment team that ranked models on leaky full-log graphs or on near-ceiling Junyi/ASSIST aggregates could therefore reach a different shortlist than one that runs this protocol first. That *decision shift*—not a claimed SOTA gain—is the applied value we target for APIN.

A key risk is that concept graphs are often inferred from the same interaction logs used to train and evaluate KT models. If graph edges are computed from the full log before train/test splitting, held-out information can leak back into the model through the graph. This leakage may occur even when the KT model itself is trained only on the training fold. Benchmarking tools such as *pyKT* [Liu et al. \(2022\)](#) have highlighted

the importance of preprocessing and split discipline, but a fold-specific audit protocol for KC graph construction remains under-specified.

A second risk arises when generic graph augmentations are imported into KT. Node dropping, edge dropping, attribute masking, and subgraph sampling were originally designed for general-purpose graphs [You et al. \(2020\)](#); [Zhu et al. \(2021\)](#); [Xia et al. \(2022\)](#); [Thakoor et al. \(2022\)](#); [Velićković et al. \(2019\)](#); [Hassani and Khasahmadi \(2020\)](#). However, prerequisite graphs are directional and educationally meaningful: removing a prerequisite edge may remove a pedagogical constraint. The degree to which label-agnostic graph augmentations disturb educational prerequisite DAGs has not been systematically measured in KT.

A third issue is the informal use of “cold-start” in KT. In this paper, we define cold-start at the KC level, based on the number of train-fold interactions associated with each KC, in contrast to user- or item-level cold-start in recommender systems [Schein et al. \(2002\)](#). This separates cold-start KC behaviour from other cold-start settings.

We address these issues at the level of protocol rather than model. We do not propose a new KT architecture or claim state-of-the-art performance. Instead, we release a leakage-controlled graph construction and audit protocol, a structural augmentation diagnostic, a ground-truth cross-validation procedure (instantiated on Junyi), and a cold-start KC report. The protocol is instantiated on Junyi Academy [Junyi Academy \(2015\)](#); [Chang et al. \(2015\)](#), ASSISTments 2012, and XES3G5M [Liu et al. \(2023b\)](#).

Novelty.

We contribute a reproducible *audit layer* around concept-graph construction—not a new KT backbone [Abdelrahman et al. \(2023\)](#); [Liu et al. \(2022\)](#); fold-specific leakage diagnostics, DDR for augmentation design on educational DAGs, KC-level cold-start strata, and ground-truth cross-validation where expert graphs exist [Bai et al. \(2025\)](#); [Chang et al. \(2015\)](#). We audit a fixed train-only inference rule and separate structural disruption from downstream AUC, which graph-KT and contrastive-KT papers typically omit [Nakagawa et al. \(2019\)](#); [Lee et al. \(2022\)](#).

Contributions.

- **C1.** A train-only protocol for constructing and auditing KC graphs with prerequisite edges and optional similarity edges $G = (\mathcal{C}, E_{\text{pre}} \cup E_{\text{sim}})$ for KT, including split checks, edge provenance, per-relation availability reporting, and DAG validation with cycle pruning.
- **C2.** The DDR metric, which quantifies how generic graph augmentations disrupt prerequisite assumptions on educational DAGs, instantiated with four formally defined label-agnostic operators and one prerequisite-preserving operator; the latter is measurably less disruptive at a matched perturbation budget, showing that DDR is a usable design target rather than a restatement of operator definitions.
- **C3.** A cold-start KC diagnostic with explicit frequency-based strata and reproducible scripts on three KT benchmarks.
- **C4.** A ground-truth cross-validation procedure that quantifies, on Junyi Academy, how train-only behavioural inference of E_{pre} agrees with the expert prerequisite

annotation of Chang et al. [Junyi Academy \(2015\)](#), using edge-level, direction-level, and reachability-level metrics across a sweep of top- K cutoffs.

- **C5.** An empirical observation, anchored on XES3G5M as the primary model-comparison benchmark, that backbone ranking under leakage-controlled graph construction is *model-specific*: GKT trails *simpleKT* by mean $\Delta\text{AUC} \approx -0.041$ on XES3G5M and ≈ -0.007 on ASSISTments (three-fold means, Table 6; descriptive fold-level 95% intervals for the XES3G5M pairs in Supplementary Table S16), while GIKT remains competitive. We pair this with a train-only versus full-log ablation and tiered leakage audits. We discuss three non-mutually-exclusive hypotheses and frame the result as a deployment and benchmarking precondition rather than as a refutation of graph-augmented KT. Supplementary Section S3 links benchmark repetitiveness to near-ceiling AUC on Junyi/ASSIST and to the near-null ablation.

Practitioner checklist.

We distill the benchmarking recommendations developed in Section 5.2 into five reporting items for graph-augmented KT papers:

- fold-specific provenance for every retained graph edge;
- cold-start KC strata alongside aggregate AUC;
- sequence-only baselines matched in compute and hyperparameter budget;
- ground-truth cross-validation when expert prerequisite annotations exist; and
- sensitivity to structure-preserving versus structure-damaging graph perturbations.

Supplementary Table S11 reports fold-level ΔAUC magnitudes and paired-test p -values; we treat the *effect sizes* in Table 6 as the primary evidence because three-fold CV limits inferential power.

Scope and benchmark roles.

We assign explicit roles to the public corpora: **primary**—XES3G5M (low KC-repeat, non-saturated AUC, clearest graph-backbone separation); **secondary**—ASSISTments 2012 (leakage ablation and cold-start); **saturated sanity**—Junyi Academy (ground-truth cross-validation and DAG density, not headline model ranking). Synthetic C2/C5 exercise tiny-graph audit modules. Model-comparison claims in C5 and Section 5.1 are anchored on XES3G5M unless noted. Table 5 reports cross-dataset diagnostic AUC; per-dataset ACC/NLL tables with 95% bootstrap confidence intervals are in Supplementary Tables S1–S5. All baselines use train-only exported graphs and valid+test sequence positions under three-fold learner CV. Protocol metrics (DDR, leakage tables, DAG audit over folds 0–2) use the same three-fold configuration unless noted. Junyi and XES3G5M include the corrected GIKT rerun in the main baseline table. DAG audit and ground-truth cross-validation prose refer to fold 0 unless noted. Similarity edges $|E_{\text{sim}}|=0$ on ASSISTments (single-skill Q-matrix) and are sparse on XES3G5M; Junyi is the only corpus where both E_{pre} and E_{sim} are populated under the default rule—we therefore describe most runs as *prerequisite-graph* protocol instances rather than as fully multi-relational deployments. On the three public benchmarks studied here, controlling the graph-leakage channel does

not by itself yield large accuracy corrections (Section 4.5): the train-only versus full-log ablation moves headline metrics by at most 0.003 AUC/ACC (synthetic sanity logs show slightly larger shifts). We therefore present leakage control as an applied reproducibility and decision-support contribution together with a measured boundary result, not as a route to higher headline numbers. We do not claim that the protocol eliminates every possible form of leakage, nor do we make claims about real-world learning outcomes.

2 Related Work

2.1 Knowledge Tracing and Benchmark Discipline

KT progressed from probabilistic models (BKT) Corbett and Anderson (1994) to deep sequence architectures (DKT, AKT, *simpleKT*) Piech et al. (2015); Ghosh et al. (2020); Liu et al. (2023a) and benchmark libraries such as pyKT Liu et al. (2022); Abdelrahman et al. (2023).

Leakage in KT and in ML more broadly. Data leakage—information that would be unavailable at prediction time influencing training or evaluation—is a long-standing failure mode in data mining Kaufman et al. (2012) and a documented driver of the reproducibility crisis across ML-based science, where Kapoor and Narayanan Kapoor and Narayanan (2023) catalogue eight leakage types and propose structured reporting. Within KT, the leakage discussion has so far centred on *label leakage* in expanded KC sequences: pyKT Liu et al. (2022) shows that one-by-one evaluation over same-question KCs inflates AUC, and Badran and Preisach Badran and Preisach (2024) demonstrate that the leakage persists during training and propose masking remedies. These works address leakage through the *interaction sequence*. The complementary channel we study—held-out information entering through the *auxiliary concept graph*—is not covered by sequence-level fixes and remains under-audited.

Limitations of this line for our setting. KT benchmarks rarely audit auxiliary graph provenance or per-knowledge component behaviour; preprocessing and split choices can shift rankings Liu et al. (2022). Our protocol standardises fold-specific graphs and cold-start reporting rather than proposing another sequence backbone.

2.2 Graph-based Knowledge Tracing and Prerequisite Structure

Graph-KT models propagate mastery over fixed or learned KC graphs (GKT, SKT, GIKT, DyGKT, DGEKT) Nakagawa et al. (2019); Tong et al. (2020); Yang et al. (2021); Cheng et al. (2024); Cui et al. (2024). Prerequisite-learning work infers directed relations from logs, content, or expert annotation Pan et al. (2017); Bai et al. (2025); Chang et al. (2015).

Limitations of this line for our setting. Graph-KT papers typically *assume* a concept graph is given and optimise predictive loss, leaving graph construction outside the experimental factorisation. Original GKT and SKT pipelines pool transition statistics over the full interaction log before learner splitting Nakagawa et al. (2019); Tong et al. (2020); our protocol isolates that construction choice from backbone design.

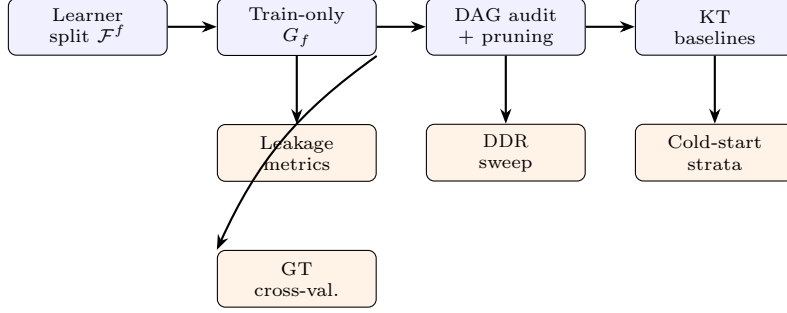


Fig. 2: Research framework connecting protocol modules (Section 3) and experiment tracks (Section 4). Fold-conditioned provenance, DAG repair, structural diagnostics decoupled from AUC, and optional expert cross-validation are the core audit outputs.

When graphs are inferred from the same log used for evaluation, held-out transitions can influence edges—a channel distinct from label leakage in the KT loss. Prerequisite-learning work optimises edge quality against expert or content signals but seldom couples inference to KT splits or reports DAG audit trails. Expert annotations (e.g. Junyi Chang et al. (2015); Junyi Academy (2015)) and behavioural inference capture related but non-identical structure; treating one as a proxy for the other is empirically unsupported without cross-validation. Our protocol keeps inference fixed and auditable, then quantifies disagreement with expert graphs where available.

2.3 Graph Augmentation, Contrastive KT, and Reproducible Resources

Graph SSL and contrastive KT import label-agnostic augmentations and GNN encoders You et al. (2020); Lee et al. (2022); Kipf and Welling (2017), but educational prerequisite DAGs impose directionality that standard drops can violate. **Limitations of this line for our setting.** Contrastive KT rarely reports acyclicity or reachability effects; KT reproducibility has focused on code and splits Liu et al. (2022) rather than graph audit logs. We release provenance artefacts, DDR sweeps, and cold-start tables to separate structural disruption from predictive gains.

Positioning. Our work is orthogonal to inventing new KT architectures or prerequisite inference rules. It supplies an audit layer—leakage diagnostics, DAG pruning logs, DDR, ground-truth cross-validation, and cold-start strata—that existing graph-KT and contrastive-KT papers typically omit.

3 Protocol

Let $\mathcal{D}$ be a KT interaction log, $\mathcal{C}$ the set of knowledge components, and $\mathcal{F} = (\mathcal{F}_{\text{train}}, \mathcal{F}_{\text{val}}, \mathcal{F}_{\text{test}})$ a learner-based temporal split. The protocol constructs a fold-specific graph $G_f = (\mathcal{C}, E_{\text{pre}}^f \cup E_{\text{sim}}^f)$ using only $\mathcal{F}_{\text{train}}^f$.

Figure 2 summarises module dependencies. Split discipline is a hard precondition: without learner-disjoint folds, downstream diagnostics are not interpretable. Graph

construction and DAG audit are coupled: train-only inference can introduce cycles, so pruning must be logged before any graph-augmented KT run. Leakage metrics, DDR, cold-start strata, and ground-truth cross-validation are *parallel* audits—they do not retrain models—which keeps the protocol lightweight but means it cannot certify every possible leakage path (e.g. hidden features in proprietary pipelines).

3.1 Formal Problem Statement and Leakage Audit

For each evaluation fold f , the protocol seeks a graph $G_f = (\mathcal{C}, \mathcal{E}_f)$ with $\mathcal{E}_f = E_{\text{pre}}^f \cup E_{\text{sim}}^f$ that is *feasible* with respect to train-only evidence and structural constraints, then verifies low direct leakage on scalar diagnostics. Let $\mathcal{I}_{\text{tr}}^f$ denote train-fold interactions and $\mathcal{H}_f = \mathcal{F}_{\text{val}}^f \cup \mathcal{F}_{\text{test}}^f$ held-out interactions.

Decision variables.

Edge sets $E_{\text{pre}}^f \subseteq \mathcal{C} \times \mathcal{C}$ and $E_{\text{sim}}^f \subseteq \mathcal{C} \times \mathcal{C}$, edge weights $w_{ij} \geq 0$, and pruning decisions on cyclic subsets of E_{pre}^f .

Objective.

The construction stage maximises retained pedagogical structure subject to sparsity and acyclicity:

$$\max_{E_{\text{pre}}^f, E_{\text{sim}}^f} \sum_{(i,j) \in E_{\text{pre}}^f} w_{ij} + \lambda \sum_{(i,j) \in E_{\text{sim}}^f} \text{sim}(c_i, c_j) \quad (1)$$

where sim is Jaccard or PMI (Section 3.4) and $\lambda \geq 0$ trades prerequisite mass against similarity mass (in practice $\lambda=0$ because the two edge types are inferred in separate pipelines).

Constraints.

$$(C1) \forall e \in \mathcal{E}_f, S_{\text{hi}}(e) = \emptyset; \quad (2)$$

$$(C2) \begin{aligned} \mathcal{U}_f^{\text{tr}} \cap \mathcal{U}_f^{\text{va}} &= \emptyset, \\ \mathcal{U}_f^{\text{tr}} \cap \mathcal{U}_f^{\text{te}} &= \emptyset, \\ \mathcal{U}_f^{\text{va}} \cap \mathcal{U}_f^{\text{te}} &= \emptyset; \end{aligned} \quad (3)$$

$$(C3) E_{\text{pre}}^f \text{ is acyclic after pruning}; \quad (4)$$

$$(C4) |E_{\text{pre}}^f| \leq K, |\text{Out}_{E_{\text{pre}}^f}(i)| \leq k; \quad (5)$$

$$\text{sim}(c_i, c_j) \geq \tau \forall (i, j) \in E_{\text{sim}}^f.$$

C1–C2 block cross-fold and cross-learner contamination; C3–C4 encode the DAG audit and top- K /top- k filters of Section 3.4. We deliberately do *not* impose $\text{TBM}_{\text{R}_f}=0$ (Eq. (9)) as a feasibility constraint: temporal-transition inference necessarily draws evidence from each learner’s full train timeline, so a zero rate would be attainable only by discarding late-train evidence. TBM_{R_f} is instead reported as a within-train stress diagnostic in the sanity/stress tier below; the released runs accordingly show non-zero values on dense corpora (Table 3).

Audit functionals (diagnostics, not training losses).

Because C1 cannot always be verified from AUC alone, the protocol reports four scalar functionals on G_f : $\text{ECR}_f^{\text{flag}}$ (Eq. (6)), $\text{ECR}_f^{\text{overlap}}$ (Eq. (7)), $|\rho_f|$ (Eq. (8)), and TBMR_f (Eq. (9)). We treat them in three tiers: **primary pass/fail**—fold-specific provenance logs, $\text{ECR}_f^{\text{flag}}=0$, and (when available) ground-truth cross-validation; **empirical bounds**—train-only versus full-log $\Delta\text{AUC}/\text{ACC}$ and cold-start strata; **sanity/stress**— $\text{ECR}_f^{\text{overlap}}$, $|\rho_f|$, and TBMR_f . Under learner-disjoint splits, $\text{ECR}_f^{\text{flag}}=0$ is expected and does not by itself certify graph quality; near-unity overlap can occur when curricula repeat across disjoint learners. TBMR_f quantifies *within-train* temporal mixing (Section 3.2), not cross-split leakage. High AUC does not certify absence of leakage; the tiered diagnostics complement discrimination scores.

3.2 Leakage Control

We use leakage to mean any information path by which held-out labels, held-out interactions, future timestamps, learner identities, or graph evidence influence graph construction, feature construction, model selection, or evaluation, although such information would not be legitimately available at the corresponding prediction point. This follows the learn–predict separation view of data leakage in data mining Kaufman et al. (2012); Kapoor and Narayanan (2023) and is especially relevant in knowledge tracing, where label leakage in expanded KC sequences can inflate reported performance Liu et al. (2022); Badran and Preisach (2024).

We audit five leakage classes at the level of *edge provenance* in KC graphs (Table 1), specialising data-mining leakage taxonomy Kaufman et al. (2012); Kapoor and Narayanan (2023) and complementing sequence-level label leakage Liu et al. (2022); Badran and Preisach (2024). Structural, edge, target, temporal, and cold-start-neighbourhood classes cover learner pooling, held-out transition support, outcome-conditioned graph artefacts, boundary violations, and tail-knowledge component edges dominated by non-train mass.

Each inferred edge is logged with its fold of origin, timestamp range, relation type, and train-only flag; severe violations halt the pipeline. In addition, we report four scalar *direct* leakage diagnostics that do not reduce to predictive accuracy. Let $\mathcal{U}_f^{\text{tr}}$, $\mathcal{U}_f^{\text{va}}$, and $\mathcal{U}_f^{\text{te}}$ denote the learner-id sets in train, validation, and test under fold f . The learner overlap flag is

$$\text{ECR}_f^{\text{flag}} := \mathbb{K}[(\mathcal{U}_f^{\text{tr}} \cap \mathcal{U}_f^{\text{va}}) \cup (\mathcal{U}_f^{\text{tr}} \cap \mathcal{U}_f^{\text{te}}) \cup (\mathcal{U}_f^{\text{va}} \cap \mathcal{U}_f^{\text{te}}) \neq \emptyset]. \quad (6)$$

When folds are learner-disjoint, $\text{ECR}_f^{\text{flag}} = 0$ for every fold—the regime enforced below.

Let $\mathcal{E}_f$ denote the graph edge set (prerequisite and similarity) consumed when evaluating fold f , and let $\mathcal{H}_f = \mathcal{F}_{\text{val}}^f \cup \mathcal{F}_{\text{test}}^f$ denote held-out interactions under that split. For edge e , let $S_{\text{hi}}(e)$ be held-out interaction tokens counted toward e ’s evidence and $\chi_{\text{ECR}^{\text{overlap}}}(e) = \mathbb{K}[|S_{\text{hi}}(e)| > 0]$. The held-out pattern overlap rate is

$$\text{ECR}_f^{\text{overlap}} = \frac{1}{|\mathcal{E}_f|} \sum_{e \in \mathcal{E}_f} \chi_{\text{ECR}^{\text{overlap}}}(e). \quad (7)$$

Table 1: Taxonomy of leakage classes audited by the protocol: typical source of contamination, a concrete KT example, and the primary direct diagnostic used in this paper (Section 3.2).

Class	Source	Example	Metric
Structural	Learner identifiers reused across train, validation, and test folds	The same learner contributes interactions to both train and held-out partitions under fold f	ECR^{flag}
Edge	Interactions outside $\mathcal{F}_{\text{train}}^f$ used as evidence for edges	Prerequisite or similarity edge retained because val/test transitions inflate co-occurrence or transition counts	$\text{ECR}^{\text{overlap}}$
Target	Outcomes or mappings pooled across splits	Q-matrix or calibration statistics fit on all learners; edge weights implicitly tuned against aggregated correctness	$ \rho $
Temporal	Violating the timestamp boundary τ_u per learner u	Post-boundary attempts included when counting $c_t \rightarrow c_{t+1}$ support or when building time-sensitive features	TBMR
Cold-start neighbourhood	Tail-knowledge component edges dominated by non-train support	Rare-knowledge component similarity edge driven mainly by interactions that belong to val/test under the split	$\text{ECR}^{\text{overlap}}$ (+ strata)

For Edge–Outcome Correlation, stack normalized edge weights into $\mathbf{w}_f \in \mathbb{R}^{|\mathcal{E}_f|}$ (for ties, use the order exported by the audit log). Let $\mathbf{y}_f \in \mathbb{R}^{|\mathcal{E}_f|}$ carry test-fold outcome summaries aligned to each edge—for example, endpoint averages of correctness on $\mathcal{F}_{\text{test}}^f$ or their logits—computed *after* graph construction (labels are used only for this diagnostic, not for inferring $\mathcal{E}_f$). The primary edge–outcome diagnostic is the Pearson correlation

$$\rho_f = \text{Corr}(\mathbf{w}_f, \mathbf{y}_f), \quad (8)$$

computed over edges with defined variance; $|\rho_f| \approx 0$ indicates that retained edge mass is statistically unrelated to test-fold outcome summaries.

Let $t_{u,i}$ be the timestamp of interaction i for learner u and let τ_u be u ’s temporal split cutoff for fold f . Write $\mathcal{I}_{u,f}^> = \{i : t_{u,i} > \tau_u\}$ for post-boundary interactions and $\mathcal{V}_{u,f}$ for the subset of those tokens that incorrectly appear in any multiset counted toward $\mathcal{E}_f$ or in graph-derived features at inference for fold f . The within-train Temporal Boundary Mixing Rate is

$$\text{TBMR}_f = \frac{\sum_u |\mathcal{V}_{u,f}|}{\sum_u |\mathcal{I}_{u,f}^>|}. \quad (9)$$

We instantiate τ_u within $\mathcal{F}_{\text{train}}^f$ at the 70% chronological split of each learner’s train sequence. TBMR_f is therefore a *within-train temporal stress statistic*: it measures how much edge evidence is supported by the late tail of each learner’s *training* sequence after an internal chronological cutoff. It must not be read as cross-fold leakage ($\text{ECR}_f^{\text{flag}}=0$ already certifies learner-disjoint splits). Elevated TBMR_f on dense multi-skill corpora (e.g. XES3G5M) flags reliance on late-train co-assessment patterns, not contamination from validation/test learners.

3.3 Graph Representation for Knowledge Tracing

We represent each fold’s knowledge structure as a directed graph $G_f = (\mathcal{C}, \mathcal{R}, \mathcal{E}_f)$ where nodes $c \in \mathcal{C}$ are knowledge components (hashed identifiers in the released pipeline), and edges carry relation type $r \in \mathcal{R} = \{\text{pre}, \text{sim}\}$ when that relation is instantiated on the corpus (prerequisite edges on all benchmarks; similarity edges only where the Q-matrix yields non-empty overlap). Prerequisite edges $(i, j) \in E_{\text{pre}}^f$ encode directed precedence inferred from temporal transitions; similarity edges $(i, j) \in E_{\text{sim}}^f$ encode co-assessment structure from the train-fold Q-matrix. Each edge stores provenance fields (fold id, train-only flag, source tag, support count, weight w_{ij}) exported as CSV for audit.

For graph-augmented KT baselines, G_f is materialised as a row-normalised adjacency matrix $A_f \in \mathbb{R}^{|\mathcal{C}| \times |\mathcal{C}|}$ (`pykt_graph_matrix.py`): prerequisite and similarity relations can be merged or consumed separately depending on the backbone (GKT uses a single KC graph; GIKT additionally uses question–KC bipartite structure from the train fold). Node features are not learned at graph-construction time; the protocol treats G_f as a *structural prior* consumed by downstream KT optimisers. This separation mirrors GNN practice on citation or knowledge graphs Kipf and Welling (2017); Hamilton et al. (2017) but with education-specific relation semantics and fold conditioning.

Technology stack: graph construction is NumPy/pandas/NetworkX; cycle detection uses Johnson’s algorithm Johnson (1975); KT training optionally delegates to pyKT Liu et al. (2022) with exported NPZ adjacency. All artefacts are versioned per dataset and fold under `data/graphs/<dataset>/fold_<f>/`.

3.4 Train-only Graph Construction

Prerequisite edges E_{pre} .

For uniformity across heterogeneous benchmarks, we infer E_{pre} from train-only temporal transitions on *all* three datasets, independently of whether an external prerequisite source is also available for that dataset. This deliberate choice decouples the audit pipeline from dataset-specific quality of expert annotations and ensures that every reported edge originates from a single, auditable inference rule. When an expert annotation is available for a benchmark (here, Junyi Academy), the protocol additionally performs ground-truth cross-validation as a separate audit step (Section 4.9).

Concretely, for each train-fold interaction sequence we extract consecutive KC transitions $c_t \rightarrow c_{t+1}$ with $c_t \neq c_{t+1}$, count their support $s(c_i, c_j)$, and retain candidate edges in three filtering stages: (i) a support-quantile filter that drops transitions below the q -th percentile (default $q = 0.95$); (ii) a per-source top- k cap (default $k = 5$) that retains, for each source KC, only the k destination knowledge components with highest support; and (iii) a global top- K cap on the number of retained candidate edges (default $K = 5,000$). Each retained edge carries the normalized support weight $w_{ij} = s(c_i, c_j)/s_{\text{max}}$ used by the cycle pruning step in Section 3.5. This candidate edge set is then passed unchanged to the DAG audit; cycle pruning operates only on the candidates returned by stages (i)–(iii).

Similarity edges E_{sim} and theoretical properties.

We compute E_{sim} from a train-only Q-matrix using either Jaccard or PMI co-occurrence at the item level (default Jaccard with threshold $\tau=0.10$). Let $I(c) \subseteq \mathcal{Q}$ be the train-fold item set tagged with concept c , with $n = |\mathcal{Q}|$ items:

$$\text{sim}_J(c_i, c_j) = \frac{|I(c_i) \cap I(c_j)|}{|I(c_i) \cup I(c_j)|}, \quad (10)$$

$$\begin{aligned} \text{sim}_{PMI}(c_i, c_j) &= \log \frac{p(c_i, c_j)}{p(c_i)p(c_j)}, \\ p(c) &= \frac{|I(c)|}{n}, \\ p(c_i, c_j) &= \frac{|I(c_i) \cap I(c_j)|}{n}. \end{aligned} \quad (11)$$

Jaccard similarity is bounded in $[0, 1]$ and yields no E_{sim} when items are single-skill (ASSISTments 2012; Section 4.8). Full operator definitions and subgraph sampling details are in Supplementary Section S4. All graph artefacts are materialised per dataset and fold.

3.5 DAG Audit

For each fold-specific prerequisite edge set, we check whether E_{pre} is acyclic. If cycles are detected, we iteratively remove the lowest-confidence edge in a detected cycle and re-run the check until a topological ordering exists. Cycle detection uses a bounded simple-cycle enumeration procedure based on Johnson’s algorithm for enumerating elementary circuits in directed graphs Johnson (1975), capped at 100 representative cycles per detection round. We use this cap because the number of elementary cycles can grow rapidly with graph density; exhaustive enumeration is unnecessary for our audit, since the pruning loop only requires representative cycle witnesses. The detector cap does not affect correctness: the pruning loop terminates only when no further cycle can be detected, so the final retained graph is acyclic. Each removed edge is written to `results/reports/<dataset>_dag_pruning_log.csv` with its source, target, confidence score, and pruning reason. The audit reports node count, raw edge count, pruned edge count, retained edge count, root count, leaf count, and final topological-sort status. The claim is not that the retained DAG is the unique correct prerequisite hierarchy, but that the pruning rule and edge provenance are explicit and reproducible.

3.6 DAG Disruption Rate and Augmentation Operators

To quantify the structural effect of graph augmentations on prerequisite DAGs, we define the DAG Disruption Rate (DDR). Let A be an augmentation operator and $A(E_{pre}, p)$ the edge set obtained at perturbation strength p . Let $\text{pre}(A(E_{pre}, p)) \subseteq E_{pre}$ be the original prerequisite edges that remain with their original direction, and let $\text{rev}(A(E_{pre}, p))$ be the edges that appear with the opposite direction. We define

$$\text{DDR}(A, p) = \frac{|E_{pre} \setminus \text{pre}(A(E_{pre}, p))| + |\text{rev}(A(E_{pre}, p))|}{|E_{pre}|}. \quad (12)$$

We evaluate four label-agnostic families from GraphCL [You et al. \(2020\)](#) (`attr_mask`, `edge_drop`, random-walk `subgraph`, `node_drop`) plus one prerequisite-preserving operator (`prereq_preserve`) that removes only transitively redundant edges at the same per-edge budget as `edge_drop` (Section 4.6; formal definitions in Supplementary Section S4). Each operator is swept over $p \in \{0.05, 0.10, 0.20, 0.30\}$ with three random seeds on each of three train folds ($3 \times 3 = 9$ runs per setting); reported means aggregate across those runs. DDR is a structural diagnostic only: it measures how much prerequisite structure is removed or reversed by an augmentation, not whether a downstream KT model becomes better or worse.

3.7 Cold-start KC Stratification

We define cold-start at the KC level, distinct from user/item cold-start in recommender systems [Schein et al. \(2002\)](#). For fold f , let $n_f(c) = |\{(u, t, q, c, y) \in \mathcal{F}_{\text{train}}^f\}|$ be the number of train-fold interactions tagged with knowledge component c . Each KC is assigned to one of four frequency strata:

$$\begin{aligned}\mathcal{S}_1 &: n_f(c) < 20, \\ \mathcal{S}_2 &: 20 \leq n_f(c) < 100, \\ \mathcal{S}_3 &: 100 \leq n_f(c) < 500, \\ \mathcal{S}_4 &: n_f(c) \geq 500.\end{aligned}$$

Strata are assigned from train-fold counts only; test rows inherit their knowledge component stratum and aggregate AUC/ACC/NLL per stratum. This exposes tail-knowledge component behaviour that aggregate metrics hide and links rare-knowledge component edge support to $\text{ECR}_f^{\text{overlap}}$ (Table 1).

3.8 Ground-Truth Cross-Validation

When a benchmark provides an expert prerequisite annotation, the protocol performs an additional, optional audit step: comparing the train-only inferred E_{pre} against the expert edge set $E_{\text{pre}}^{\text{expert}}$. The procedure has four stages: (i) load the expert annotation and filter rows by a prerequisite-score threshold θ to obtain a directed expert edge set; (ii) align expert KC identifiers to the pipeline’s hashed `kc_id` space via a deterministic name-to-id mapping reconstructed from the preprocessing hash function; (iii) sweep top- K cutoffs over the inferred edge set and compute, at each K : edge precision/recall/F1, direction agreement, reachability F1 (over transitive closure), and node Jaccard; (iv) extract qualitative examples of disagreement in three categories: edges present only in inferred, edges present only in expert, and edges with opposite direction. The procedure is read-only with respect to the pipeline; no inferred edge is added or removed based on the comparison.

Table 2: Dataset statistics used in the P0 diagnostic protocol. Junyi Academy counts reflect the Chang et al. problem-level log (`junyi.ProblemLog_original.csv`) after preprocessing; #items and #KCs coincide because both map to the exercise column. Synthetic C2/C5 are companion sanity logs shipped with the preprocessing scripts.

Dataset	# learners	# items	# KCs	# interactions	Avg. seq. len.	Has DAG
Junyi Academy	247,606	722	722	25,925,992	104.7	Yes
ASSISTments 2012	29,018	53,086	265	2,711,602	93.4	No
XES3G5M	18,066	7,652	865	6,413,353	355.0	Yes
Synthetic C2	4,000	50	2	200,000	50.0	No
Synthetic C5	4,000	50	5	200,000	50.0	No

4 Experiments

4.1 Datasets

Benchmark roles.

XES3G5M Liu et al. (2023b) is our *primary* public benchmark for model-comparison claims: low consecutive KC-repeat ($\approx 21\%$, Supplementary Section S3), non-saturated headline AUC (≈ 0.875 for *simpleKT*), and the widest separation among graph backbones under train-only graphs. **ASSISTments 2012** is *secondary*: it stress-tests leakage ablation, cold-start strata, and DAG audit on a single-skill Q-matrix ($|E_{\text{sim}}|=0$). **Junyi Academy** Junyi Academy (2015) is a *saturated sanity* corpus for ground-truth cross-validation and dense E_{pre} topology; near-ceiling AUC (≈ 0.98 – 0.984) makes it unsuitable for headline graph-benefit claims. Synthetic C2/C5 are controlled tiny-graph sanity checks.

We instantiate the protocol on all five corpora. Junyi ships an expert-curated prerequisite annotation at the exercise-pair level. *In the main inference branch we do not consume this expert DAG:* E_{pre} for Junyi is built by the same train-only temporal inference rule used for ASSISTments and XES3G5M (Section 3.4). The expert DAG enters only the read-only ground-truth cross-validation sub-protocol (Section 4.9). ASSISTments provides a Q-matrix but no external prerequisite graph. XES3G5M provides hierarchical KC metadata parsed at the sequence level; E_{pre} is inferred by the shared train-only rule. Table 2 summarises the preprocessed datasets. The public benchmarks differ substantially in interaction volume, KC vocabulary size, and item-to-KC granularity. This heterogeneity is useful for auditing because the protocol must remain leakage-controlled under both curated and inferred graph settings. We use learner-based temporal splits with ratios (0.7, 0.1, 0.2) and seed 42; all edge provenance is regenerated for each run.

Benchmark repetitiveness.

Learner-disjoint splits remove cross-fold leakage but not within-log repetition short-cuts. Junyi and ASSISTments exhibit high consecutive KC-repeat rates (0.66–0.76), whereas XES3G5M repeats only 21% of the time (Supplementary Section S3). On the

Table 3: Direct leakage diagnostics per dataset (fold means). ECR_{flag} : learner-overlap indicator (Eq. 6). $\text{ECR}_{\text{overlap}}$: held-out pattern overlap (Eq. 7). EOC: exported monotone transform of edge–outcome Pearson $|\rho|$ (Eq. 8; $|\rho| \approx 0.09\text{--}0.22$ on public corpora). TBMR: within-train temporal mixing (Eq. 9), not a train/test violation.

Dataset	ECR_{flag}	$\text{ECR}_{\text{overlap}}$	EOC	TBMR
ASSISTments 2012	0.000	1.000	1.423	0.201
Junyi Academy	0.000	1.000	1.449	0.176
Synthetic C2	0.000	1.000	0.000	0.733
Synthetic C5	0.000	1.000	0.000	0.133
XES3G5M	0.000	0.988	1.420	0.756

Table 4: DAG audit summary (fold 0). *Raw* and *Final* are the prerequisite edge counts before and after lowest-confidence cycle pruning inside `dag_audit`; *Pruned* is the difference. The *Cycles* column reports representative cycles found before pruning (cap 100). All corpora pass topological sort with zero cycles after pruning on fold 0.

Dataset	$ V $	Raw $ E_{\text{pre}} $	Pruned	Final $ E_{\text{pre}} $	Roots	Leaves	Cycles (cap)
Junyi Academy	573	2,152	1,013	1,139	90	92	≥ 100
ASSIST’12	139	413	152	261	34	23	≥ 100
XES3G5M	455	1,162	461	701	129	73	≥ 100
Synthetic C2	2	2	1	1	1	1	1
Synthetic C5	3	2	0	2	1	1	0

repetition-heavy corpora, *simpleKT* reaches $\text{AUC} \approx 0.968\text{--}0.984$ despite weak edge–outcome correlation (Table 3), whereas XES3G5M remains near 0.875 under the same protocol. This pattern inflates absolute deep-KT AUC where KC repetition is high, helps explain the near-null train-only versus full-log ablation (Section 4.5), and is why we anchor model-comparison claims on XES3G5M rather than on Junyi/ASSIST headline AUC.

4.2 Leakage and DAG Audit

Table 3 summarises the direct leakage metrics on the released runs. ECR_{flag} is identically zero on every dataset (fold means), confirming learner-disjoint splits and structural leakage freedom at the evaluation protocol level; $\text{ECR}_{\text{overlap}}$ stays near unity because recurrent transition patterns reappear among held-out learners even though edges are inferred strictly from train folds. The edge–outcome correlation is weak on all three public corpora ($|\rho| \approx 0.09\text{--}0.22$), so retained edge mass is only weakly aligned with test-fold outcome summaries; ρ is undefined on degenerate synthetic graphs where edge-weight variance vanishes. TBMR is reported as the within-train temporal stress statistic of Section 3.2 (the share of edge evidence drawn from the late, post- τ_u tail of each learner’s train sequence); it ranges from 0.18 on Junyi to 0.75 on XES3G5M. Because folds are learner-disjoint ($\text{ECR}_{\text{flag}}=0$), these values quantify reliance on late train transitions rather than any cross-split contamination. We read $\text{ECR}_{\text{flag}}=0$ and

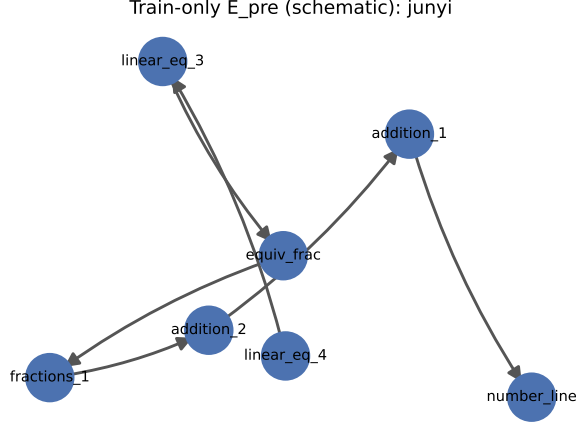


Fig. 3: Schematic visualisation of a high-support prerequisite chain in train-only E_{pre} on Junyi Academy after DAG audit (representative subgraph; the full graph contains hundreds to thousands of edges). ASSISTments 2012 and XES3G5M panels appear in Supplementary Figure S1.

provenance logs as **primary** pass/fail checks; overlap, ρ , and TBMR as **sanity/stress** indicators that can be near-degenerate under honest splits (Section 5.3). The leakage audit reports that all retained graph edges are train-only under the current fold configuration. Table 4 reports the DAG audit on fold 0 for all five corpora. The protocol produces non-empty acyclic prerequisite graphs after lowest-confidence cycle pruning on every dataset except the degenerate Synthetic C2 case ($|E_{\text{pre}}|=1$ after pruning).

On Junyi fold 0, 2,152 candidate edges prune to 1,139 acyclic edges; ASSISTments and XES3G5M retain 261 and 701 edges respectively after pruning (Table 4). Even train-only inference can induce cycles; the audit makes the pruning rule explicit rather than silently fixing the graph.

Figure 3 illustrates the *shape* of retained prerequisite structure on the densest public benchmark. These plots are not used for inference; they help reviewers compare graph density with DDR and baseline behaviour in later sections.

4.3 Controlled test-fold edge injection

To verify that the audit metrics respond to graph-mediated leakage, we deliberately contaminate the XES3G5M fold-0 train-only prerequisite builder with 5% and 20% of test-fold interactions (Supplementary Table S17). Candidate-edge count ($|E_{\text{pre}}|$: 1162→1378→1417; counted *before* DAG pruning, whereas Table 4 reports the 701 edges retained *after* pruning on the clean fold) and TBMR_f (0.754→0.776→0.779) increase monotonically with the injection rate, while $\text{ECR}_f^{\text{flag}}$ remains 0 because learner-id sets stay disjoint: flag-level checks alone cannot detect this contamination channel, which motivates the tiered audit. $\text{ECR}_f^{\text{overlap}}$ is already saturated at 1.0 on this transition-dense corpus (Section 3.1), so it carries no additional signal here.

Table 5: Diagnostic baseline AUC summary (train-only graphs; three-fold learner CV means). XES3G5M is the primary model-comparison column; Junyi is a saturated sanity corpus. Baselines sanity-check preprocessing, splits, and pyKT wiring; they are not SOTA claims. Per-dataset ACC/NLL and 95% bootstrap confidence intervals are in Supplementary Tables S1–S5.

Model	XES 3G5M	ASSIST 2012	Junyi	Synth C2	Synth C5
BKT	0.636	0.669	0.780	0.835	0.791
AKT	0.875	0.968	0.984	0.873	0.822
DKT	0.858	0.963	0.984	0.862	0.826
<i>simpleKT</i>	0.875	0.968	0.984	0.875	0.829
GKT	0.834	0.961	—	0.873	0.810
GIKT	0.878	0.967	0.982	0.875	0.820
SKT	0.625	0.953	0.979	0.708	0.810
DyGKT	0.752	0.952	0.978	0.710	0.826
DGEKT	0.841	0.960	0.981	0.710	0.830

4.4 Baseline Diagnostic

Table 5 sanity-checks preprocessing, learner splits, train-only graph export, and pyKT wiring across five corpora; it is not a SOTA claim. We read the table through the benchmark roles of Section 4.1. On the *primary* benchmark XES3G5M, sequence-only *simpleKT*/AKT (≈ 0.875) lead; GKT trails by ≈ 0.041 AUC; SKT and DyGKT collapse (≈ 0.625 – 0.752); corrected GIKT slightly exceeds the sequence leader (0.878 vs. 0.875). On ASSISTments (secondary), GKT trails by ≈ 0.007 AUC with near-ceiling scores for all leaders. On saturated Junyi, all deep models cluster near 0.98–0.984 and are poor discriminators of graph benefit. Per-dataset ACC/NLL and bootstrap intervals are in Supplementary Tables S1–S5.

Training parity.

Full hyperparameter parity is in Supplementary Table S15. Sequence-only checkpoints (AKT, DKT, *simpleKT*) use stock pyKT configurations. Native graph backbones (GIKT, SKT, DyGKT, DGEKT) share the same processed splits, data loader, evaluation hook, learning rate, split seed, and early-stopping rule (patience 5, best validation AUC) as the pyKT runs, and no model family receives a hyperparameter search. Epoch caps and batch sizes are *not* identical across families: sequence checkpoints train for up to 30 epochs (batch 64) while graph backbones use 10-epoch budgets with smaller batches (Table S15), reflecting their higher per-epoch cost. Checkpoint selection by best validation AUC mitigates, but does not eliminate, this residual budget difference; we flag it as a caveat on the GKT deficit in Section 5.1. GKT is evaluated through the pyKT graph branch on corpora where that wrapper is supported.

Table 6 reports three-fold learner-CV means $\pm$ std and Wilcoxon p -values vs. *simpleKT*. With only three paired folds the Wilcoxon test is underpowered (minimum two-sided $p=0.25$); paired t -test p -values on the same fold AUCs appear in Supplementary Table S11. The resulting *model-specific* ordering—GKT as the consistent under-performer, GIKT competitive or slightly above sequence leaders—is developed in Section 5.1.

Table 6: Three-fold learner CV baseline summary (train-only graphs, valid+test positions). Mean $\pm$ std over folds; p_W : Wilcoxon signed-rank test vs. *simpleKT* (paired fold AUCs).

Dataset	Model	AUC	ACC	NLL	p_W
ASSIST '12	<i>simpleKT</i>	0.968 $\pm$ 0.000	0.916 $\pm$ 0.001	0.173 $\pm$ 0.001	—
	GKT	0.961 $\pm$ 0.001	0.907 $\pm$ 0.001	0.185 $\pm$ 0.001	0.250
	GIKT	0.967 $\pm$ 0.000	0.915 $\pm$ 0.001	0.175 $\pm$ 0.001	0.250
	SKT	0.953 $\pm$ 0.001	0.898 $\pm$ 0.001	0.198 $\pm$ 0.001	0.250
	DyGKT	0.952 $\pm$ 0.001	0.899 $\pm$ 0.001	0.200 $\pm$ 0.002	0.250
	DGEKT	0.960 $\pm$ 0.000	0.907 $\pm$ 0.001	0.186 $\pm$ 0.001	0.250
XES3G5M	<i>simpleKT</i>	0.875 $\pm$ 0.000	0.858 $\pm$ 0.001	0.324 $\pm$ 0.001	—
	GKT	0.834 $\pm$ 0.001	0.842 $\pm$ 0.000	0.364 $\pm$ 0.001	0.250
	GIKT	0.878 $\pm$ 0.000	0.860 $\pm$ 0.000	0.321 $\pm$ 0.001	0.250
	SKT	0.625 $\pm$ 0.001	0.800 $\pm$ 0.001	0.482 $\pm$ 0.001	0.250
	DyGKT	0.752 $\pm$ 0.001	0.813 $\pm$ 0.001	0.431 $\pm$ 0.001	0.250
	DGEKT	0.841 $\pm$ 0.000	0.844 $\pm$ 0.000	0.358 $\pm$ 0.001	0.250
Junyi Academy	<i>simpleKT</i>	0.984 $\pm$ 0.003	0.951 $\pm$ 0.008	0.121 $\pm$ 0.019	—
	GIKT	0.982 $\pm$ 0.000	0.946 $\pm$ 0.000	0.133 $\pm$ 0.001	1.000
	SKT	0.979 $\pm$ 0.005	0.938 $\pm$ 0.011	0.139 $\pm$ 0.023	0.250
	DyGKT	0.978 $\pm$ 0.000	0.934 $\pm$ 0.000	0.151 $\pm$ 0.001	0.250
	DGEKT	0.981 $\pm$ 0.000	0.943 $\pm$ 0.000	0.137 $\pm$ 0.001	0.250

Table 7: Train-only versus full-log graph ablation summary (fold 0). Columns report the largest absolute Δ AUC and Δ ACC (full-log minus train-only) across GKT, GIKT, SKT, DyGKT, and DGEKT. Per-model train-only/full-log pairs are in Supplementary Tables S6–S10.

Dataset	max $ \Delta$ AUC—	max $ \Delta$ ACC—	Largest shift (model)
Junyi Academy	0.003	0.008	SKT (Δ AUC=−0.003)
ASSISTments 2012	0.000	0.000	all models $\approx$ 0
XES3G5M	0.003	0.001	DyGKT (Δ AUC=−0.003)
Synthetic C2	0.001	0.001	GKT/DGEKT
Synthetic C5	0.002	0.002	GKT (Δ AUC=+0.002); GIKT (Δ ACC=−0.002)

4.5 Graph Construction Ablation (Train-only vs. Full Log)

We contrast fold-specific train-only graph exports (`e\_pre\_train\_only.csv`, `e\_sim\_train\_only.csv`) against a *single* graph inferred from the entire preprocessed log (`data/processed/<dataset>/full\_log/`), holding learner splits and evaluation code fixed. Only the static graph artefact fed into the pyKT graph branch differs; KT optimisation still uses train-fold sequences only.

Table 7 summarises the largest train-only versus full-log shifts across GKT, GIKT, SKT, DyGKT, and DGEKT (fold 0). On the three public benchmarks, $|\Delta$ AUC— and $|\Delta$ ACC stay at or below 0.003; Synthetic C5 shows the largest released shift ($|\Delta$ AUC|=0.002 for GKT). Per-model pairs appear in Supplementary Tables S6–S10. Under our support quantile ($q=0.95$), top- k cap, and $K=5000$ filter, pooling learners

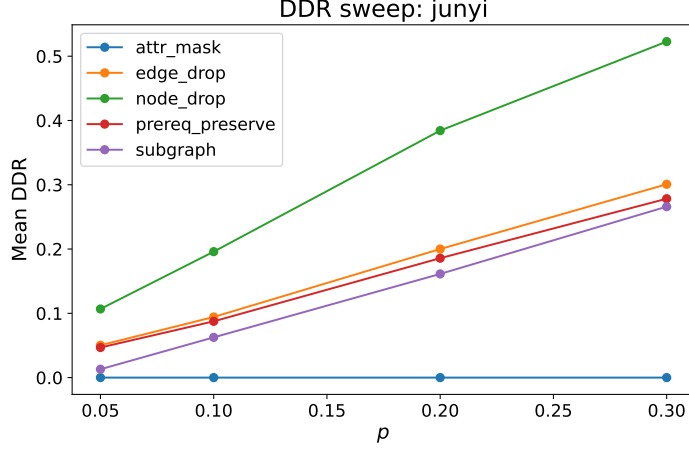


Fig. 4: DDR as a function of perturbation strength p on Junyi Academy for five graph augmentation families: four label-agnostic operators (`attr_mask`, `subgraph`, `edge_drop`, `node_drop`) and one prerequisite-preserving operator (`prereq_preserve`). Each marker plots the mean DDR averaged across nine stochastic runs (three random seeds on each of three train folds). The same ordering holds on ASSISTments 2012 and XES3G5M (Supplementary Figure S2): $\text{DDR}(\text{attr_mask}) = 0 < \text{DDR}(\text{subgraph}) \lesssim \text{DDR}(\text{prereq_preserve}) < \text{DDR}(\text{edge_drop}) \approx p < \text{DDR}(\text{node_drop})$.

before edge inference rarely changes surviving transitions on public benchmarks. Train-only construction is therefore *not* penalising headline metrics relative to this deliberate full-log variant on those corpora, without proving absence of leakage in general.

4.6 DAG Disruption Rate

Figure 4 and Table 8 show five consistent regimes across the three benchmarks. First, attribute masking leaves E_{pre} structurally unchanged and therefore gives $\text{DDR} = 0$ across perturbation strengths by construction. Second, random-walk subgraph sampling preserves connectivity inside the sampled subgraph and disrupts E_{pre} *sub-linearly* in p : because edges are removed only at the boundary of the visited subgraph, the disruption is consistently smaller than under edge dropping at the same p (e.g. on Junyi, $\text{DDR} = 0.16$ for subgraph vs. 0.20 for edge drop at $p = 0.20$). Third, edge dropping disrupts prerequisite structure approximately linearly in p , as expected when each edge is removed independently. Fourth, node dropping produces the largest disruption because removing a node also removes all incident prerequisite edges; at $p = 0.30$, node dropping disrupts roughly half of the prerequisite structure on all three benchmarks ($\text{DDR} \approx 0.52\text{--}0.54$).

Table 8: Mean DDR across three seeds and three train folds for the five augmentation families and four perturbation strengths. Subgraph sampling is random-walk based (Section 3.6); `prereq_preserve` is the prerequisite-preserving operator that, at the same per-edge budget as `edge_drop`, removes only transitively redundant edges and protects the transitive-reduction backbone. Lower is less disruptive; 0.000 for attribute masking is by construction.

Dataset	Family	$p=0.05$	$p=0.10$	$p=0.20$	$p=0.30$
Junyi Academy	<code>attr_mask</code>	0.000	0.000	0.000	0.000
	<code>subgraph</code>	0.013	0.063	0.161	0.266
	<code>prereq_preserve</code>	0.047	0.088	0.186	0.278
	<code>edge_drop</code>	0.050	0.094	0.200	0.301
	<code>node_drop</code>	0.107	0.196	0.384	0.523
ASSISTments 2012	<code>attr_mask</code>	0.000	0.000	0.000	0.000
	<code>subgraph</code>	0.012	0.028	0.071	0.188
	<code>prereq_preserve</code>	0.048	0.087	0.179	0.262
	<code>edge_drop</code>	0.055	0.100	0.212	0.309
	<code>node_drop</code>	0.092	0.193	0.389	0.530
XES3G5M	<code>attr_mask</code>	0.000	0.000	0.000	0.000
	<code>subgraph</code>	0.022	0.058	0.145	0.240
	<code>prereq_preserve</code>	0.038	0.072	0.158	0.233
	<code>edge_drop</code>	0.049	0.095	0.204	0.299
	<code>node_drop</code>	0.110	0.196	0.386	0.535

DDR is a usable design target, not a tautology.

A natural objection to the four label-agnostic families is that their DDR ordering merely re-states their definitions: an operator that deletes more edges disrupts more structure. To test whether DDR *discriminates* structure-aware from structure-agnostic augmentation rather than re-describing edge counts, we add a fifth, prerequisite-preserving operator (`prereq_preserve`). It draws the same per-edge budget p as `edge_drop`, but only removes edges that are *transitively redundant*—those whose endpoints remain connected through an alternative directed path—so that every reachability/precedence relation and the entire transitive-reduction backbone are preserved. Crucially, unlike `attr_mask`, this operator is *not* DDR=0 by construction: its disruption is an empirical quantity equal to p times the redundant-edge fraction of the graph. At every dataset and strength, `prereq_preserve` is strictly less disruptive than `edge_drop` at a matched budget (at $p = 0.30$, 0.278 vs. 0.301 on Junyi, 0.262 vs. 0.309 on ASSISTments, and 0.233 vs. 0.299 on XES3G5M). The size of this gap is *data-dependent*—a $\sim 8\%$ relative reduction on Junyi but $\sim 22\%$ on XES3G5M—and is determined by graph redundancy rather than by the operator definition. Because temporal-precedence inference generates many transitive shortcuts (if a precedes b precedes c , the edge $a \rightarrow c$ is also counted), the inferred graphs are highly redundant ($\approx 78\text{--}92\%$ of edges lie off the transitive reduction), which both explains the modest gap and shows that DDR responds to operator structure-awareness in a quantitative, non-tautological way. DDR can therefore serve as a concrete design target: an augmentation that respects prerequisite directionality is measurably rewarded with lower disruption at the same perturbation budget.

Table 9: DDR vs. downstream graph-KT accuracy. For each dataset and model, the prerequisite graph E_{pre} is perturbed by each operator at strength p ; *AUC drop* is the mean decrease in test AUC relative to the unperturbed (DDR= 0) baseline graph across folds. Pooled correlations (DDR vs. AUC drop): assist2012/dgekt: $r=0.15$ ($p=4.7\text{e-}01$), $\rho=0.18$; xes3g5m/dgekt: $r=0.13$ ($p=5.1\text{e-}01$), $\rho=0.18$.

Dataset	Model	Operator	p	Mean DDR	Mean AUC	AUC drop
assist2012	dgekt	edge_drop	0.10	0.086	0.9600	0.0000
assist2012	dgekt	edge_drop	0.20	0.221	0.9601	-0.0000
assist2012	dgekt	edge_drop	0.30	0.304	0.9600	-0.0000
assist2012	dgekt	node_drop	0.10	0.171	0.9601	-0.0001
assist2012	dgekt	node_drop	0.20	0.369	0.9601	-0.0000
assist2012	dgekt	node_drop	0.30	0.493	0.9601	-0.0000
assist2012	dgekt	prereq_preserve	0.10	0.080	0.9601	-0.0001
assist2012	dgekt	prereq_preserve	0.20	0.206	0.9601	-0.0000
assist2012	dgekt	prereq_preserve	0.30	0.278	0.9600	0.0000
xes3g5m	dgekt	edge_drop	0.10	0.093	0.8417	0.0001
xes3g5m	dgekt	edge_drop	0.20	0.214	0.8416	0.0001
xes3g5m	dgekt	edge_drop	0.30	0.301	0.8417	0.0000
xes3g5m	dgekt	node_drop	0.10	0.177	0.8417	0.0001
xes3g5m	dgekt	node_drop	0.20	0.424	0.8414	0.0003
xes3g5m	dgekt	node_drop	0.30	0.555	0.8417	0.0001
xes3g5m	dgekt	prereq_preserve	0.10	0.071	0.8417	0.0001
xes3g5m	dgekt	prereq_preserve	0.20	0.168	0.8417	0.0001
xes3g5m	dgekt	prereq_preserve	0.30	0.233	0.8415	0.0003

The relative ordering $\text{DDR}(\text{attr_mask}) < \text{DDR}(\text{subgraph}) \lesssim \text{DDR}(\text{prereq_preserve}) < \text{DDR}(\text{edge_drop}) < \text{DDR}(\text{node_drop})$ is consistent across all three benchmarks and all four perturbation strengths. We read these results as a structural diagnostic, not as a claim that any augmentation is beneficial or harmful for a downstream KT learner. They indicate that prerequisite-aware augmentation design is needed when graph augmentations are applied to educational DAGs: node-level operators are an order of magnitude more disruptive than attribute-level operators, with subgraph sampling and the prerequisite-preserving operator occupying an intermediate, structure-preserving regime.

4.7 DDR Downstream Stress Test

Table 9 connects DDR to an *exploratory* DGEKT-only retraining stress test on ASSISTments 2012 and XES3G5M (three folds; **edge_drop**, **node_drop**, **prereq_preserve**). Despite large structural disruption (**node_drop** mean DDR 0.493–0.555 at $p=0.30$), downstream AUC changes for *DGEKT* stay near zero and pooled DDR–AUC-drop correlations are weak ($r=0.13$ – 0.15 ; supplementary DDR–downstream figure). DDR therefore audits augmentation *structure*; it is not, on the evidence reported here, a universal proxy for accuracy degradation across graph-consuming backbones.

Table 10: Cold-start KC diagnostic summary. Column n counts fold 0 test interactions per stratum for *simpleKT*; $\Delta_{\max}$ is the best graph-family AUC minus the best sequence-only AUC (three-fold means). Per-model strata appear in Supplementary Tables S11–S12.

Dataset	Stratum	n	<i>simpleKT</i> AUC	$\Delta_{\max}$
Junyi Academy	very_cold	19	1.000	0.000
	cold	256	0.447	0.000
	warm	11,257	0.615	-0.004
	hot	$\approx 7.7 \times 10^6$	0.718	-0.006
ASSIST'12	very_cold	49	0.608	+0.002
	cold	707	0.769	0.000
	warm	9,017	0.765	+0.001
	hot	$\approx 8.1 \times 10^5$	0.693	-0.005
XES3G5M	very_cold	161	0.810	+0.005
	cold	2,592	0.798	0.000
	warm	17,655	0.790	-0.007
	hot	$\approx 1.9 \times 10^6$	0.717	-0.009
Synthetic C2	hot	60,000	0.624	+0.008
Synthetic C5	hot	60,000	0.630	+0.002

4.8 Cold-start KC Diagnostic

Table 10 shows that aggregate AUC can hide per-stratum variation: on real benchmarks the best graph-aware family leads only in *very_cold* ($\Delta_{\max}=0.002$ on ASSISTments, +0.005 on XES3G5M) and is tied or worse in *warm/hot* (mean $\Delta_{\max} \approx -0.005$). Tiny very-cold counts ($n < 200$ on Junyi/ASSIST) should not be over-interpreted. Synthetic C2/C5 route all test mass to *hot*; their positive $\Delta_{\max}$ values reflect clean generator structure, not cold-start relief. GKT underperforms sequence leaders on synthetic hot rows, reinforcing model-specific graph benefit.

Similarity edges and dataset properties.

The similarity-edge rule of Section 3.4 is sensitive to the multi-skill density of the Q-matrix. On the train fold of ASSISTments 2012, 100% of items are tagged with exactly one KC, so $|I(c_i) \cap I(c_j)| = 0$ for every distinct concept pair, and both Jaccard and PMI yield no edges. On XES3G5M, 86.6% of train items are single-skill but the remaining multi-skill items generate 354 similarity edges at the default Jaccard threshold of 0.10. We treat this dataset-property dependency as a first-class outcome of the audit: a similarity rule appropriate for multi-skill benchmarks may need to be replaced or supplemented (e.g., by response-pattern correlation) on single-skill benchmarks. This is not a pipeline failure but a structural observation that protocol papers should report.

4.9 Ground-Truth Cross-Validation on Junyi

Junyi Academy [Junyi Academy \(2015\)](#) is the only benchmark in our study that ships with a publicly released expert prerequisite annotation (Chang et al.). The annotation enumerates directed exercise-pair relations with a continuous `Prerequisite_avg` score on a mean-opinion-score-like scale. We use $\theta = 0.5$ as the prerequisite filter,

retaining 1,131 directed expert edges over 361 distinct exercise endpoints that participate in those edges. Following Section 3.8, expert exercise names are mapped to pipeline `kc_id` via the preprocessing hash function reconstructed from `data/raw/junyi/junyi_Exercise_table.csv`. All 1,131 expert edges align deterministically: the alignment audit reports `matched = 1,131`, `missingsrc = 0`, `missingdst = 0`, and alignment rate 1.0. We compare this expert edge set against the fold 0 train-only inferred list in `e\_pre\_train\_only.csv`, which contains 2,152 ranked directed candidates (the same file audited in Section 4.2; cycle pruning yields 1,139 retained acyclic edges).

Table 11: GT cross-validation: agreement between train-only inferred E_{pre} and expert prerequisite DAG on Junyi Academy ($|E| = 1131$ directed edges).

K (top inferred)	Edge prec.	Edge rec.	Dir. agr.	Rch. F1
200	0.235	0.042	0.346	0.008
500	0.178	0.079	0.225	0.021
800	0.181	0.128	0.280	0.062
$ E_{\text{expert}} = 1131$	0.183	0.183	0.261	0.104
1200	0.180	0.191	0.262	0.126
1800	0.153	0.243	0.278	0.260
2500	0.143	0.271	0.285	0.339
3500	0.143	0.271	0.285	0.339
5000 (cap)	0.143	0.271	0.285	0.339

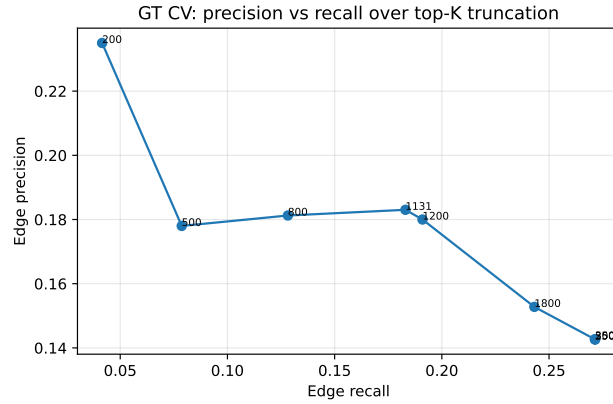


Fig. 5: Edge-level precision and recall of train-only inferred E_{pre} versus the expert prerequisite annotation on Junyi, across top- K cutoffs $K \in \{200, 500, 800, 1131, 1200, 1800, 2500, 3500, 5000\}$. The curve plateaus at $K \geq 2,500$ because the inferred edge pool tops out at 2,152 after DAG audit.

Table 11 and Figure 5 show low edge-level overlap ($F1 \approx 0.18$ at $K=|\text{expert}|$) that plateaus as K grows, modest direction agreement (≈ 0.26 – 0.29 on shared pairs), but comparatively high node Jaccard (0.632 at $K=|\text{expert}|$; Supplementary Table S14 reports the full top- K sweep). Reachability F1 rises to 0.339 at large K , indicating recovery of indirect chains despite edge disagreement. Behavioural inference therefore encodes co-occurrence more than pedagogical ordering; it is not a low-cost substitute for expert curation and should be combined as a separate relation when both sources exist. Qualitative disagreement examples appear in Supplementary Section S5. The audit uses $\theta=0.5$ on a single seed and was not swept over stricter thresholds.

5 Discussion

5.1 Model-specific graph benefit under leakage-controlled evaluation

The baseline summary (Table 5) and fold-level diagnostics (Table 6) support an empirical observation at the level of the protocol itself. We anchor the discussion on **XES3G5M** (primary; Section 4.1) because its lower KC-repeat and non-saturated AUC make backbone separation interpretable. Under leakage-controlled evaluation (train-only E_{pre} , optional train-only E_{sim} where populated, learner-based temporal split), explicit concept-graph benefit is *model-specific* rather than uniform. On XES3G5M, *simpleKT* averages 0.875 AUC while GKT averages 0.834 (mean $\Delta \approx -0.041$ across three folds; fold stds in Table 6). SKT (0.625) and DyGKT (0.752) trail far more severely. GIKT (0.878) slightly *exceeds* the sequence leader ($+0.003$ mean AUC). On ASSISTments (secondary), the same ordering direction holds but magnitudes shrink (GKT trails by ≈ 0.007 AUC with near-ceiling leaders). On saturated Junyi, all models cluster within ≈ 0.002 – 0.006 AUC of *simpleKT* and should not drive deployment conclusions. We report Supplementary Table S11 p -values for completeness but treat *fold-level ΔAUC magnitudes* as the primary evidence because three-fold CV yields underpowered inferential tests ($p_{\min}=0.25$ for Wilcoxon; fragile t -tests on $n=3$ folds).

We do not claim that graph-augmented KT is uninformative; the protocol removes a specific channel through which held-out information can reach the model—the concept graph itself. Three non-mutually-exclusive hypotheses are consistent with the observed ordering.

H1 – Leakage in prior evaluations.

A common practice in graph-augmented KT is to compute graph statistics from the full interaction log prior to train/test splitting. Under such practice, graph-augmented models can observe test-fold transitions through the graph during training. Our protocol enforces fold-specific edge inference and provenance logging. Table 7 instantiates the controlled comparison: identical learner splits and evaluation code, but graph statistics pooled over the full preprocessed log. Headline metrics on the three public benchmarks shift by at most $|\Delta\text{AUC}| \leq 0.003$; Synthetic C5 shows a modest shift ($|\Delta\text{AUC}|=0.002$). H1 remains relevant as a *risk hypothesis* about workflows that leak

held-out mass into graph construction; it is not a causal claim that we have confirmed on these benchmarks. Table 7 caps how much headline leakage can explain the observed GKT deficit here ($|\Delta\text{AUC}| \leq 0.003$ on public corpora). Architectures that exploit cross-split pooling more aggressively might show larger shifts.

H2 – Aggregate-metric masking.

The cold-start diagnostic (Section 4.8) shows that per-stratum behaviour can differ from aggregate behaviour. Table 10 makes this concrete: on real data, the best graph-aware model is slightly better in the *very_cold* stratum, essentially tied in *cold*, and worse in *warm/hot*. On Synthetic C2/C5, where all test interactions are hot, GIKT/DGEKT improve over sequence-only baselines, but this reflects clean synthetic structure rather than cold-start relief.

H3 – Strong-backbone substitution.

Modern attention-based backbones such as AKT and *simpleKT* may already capture much relational information through long-range temporal attention. Additional structural signal at the concept-embedding level may then yield diminishing returns—a pattern weaker temporal backbones should benefit from more, all else equal.

These hypotheses are not exhaustive. Distinguishing among them requires re-running prior graph-KT papers under our protocol, which is beyond this paper’s scope. The narrower observation is that *under leakage-controlled evaluation, the ranking between graph-augmented and sequence-only KT can differ from settings that pool graph evidence across splits*, and benchmarking practice should treat graph provenance as a first-order factor.

Why this strengthens the protocol.

If graph-augmented KT reliably outperformed sequence-only baselines regardless of graph provenance, leakage control would be secondary. The observation that sequence-only pyKT checkpoints match or exceed GKT on XES3G5M (primary) and ASSISTments under train-only graphs, while corrected GIKT remains competitive on XES3G5M without a train-only/full-log gap, together with edge provenance auditing, elevates leakage control from hygiene to a deployment and benchmarking precondition. Combined with Section 4.9, which shows that inferred and expert prerequisite signals are structurally distinct, graph quality cannot be assumed—it must be audited.

Caveats.

Baseline ordering uses three-fold means; DAG audit and GT cross-validation refer to fold 0 exports by default. GKT, AKT, DKT, and *simpleKT* are stock pyKT Liu et al. (2022) checkpoints; GIKT, SKT, DyGKT, and DGEKT are native implementations in our fork (`src/models/`). Epoch caps differ across families (30 for sequence checkpoints vs. 10 for graph backbones; Table S15): although all models select the best-validation-AUC checkpoint under the same patience, the smaller graph-model budget is a residual training-budget confound for the GKT deficit, and the severe SKT/DyGKT deficits

in particular may partly reflect under-tuned native implementations rather than backbone limits. Stronger graph-KT architectures [Tong et al. \(2020\)](#); [Cheng et al. \(2024\)](#); [Cui et al. \(2024\)](#) may behave differently; auditing them is an explicit follow-up.

5.2 Broader implications

Scalability and broader domains.

The protocol is dataset-agnostic: cost scales with train-fold transition counting and capped cycle enumeration, not KT training. Sparsity caps keep audit tractable on million-scale logs [Choi et al. \(2020\)](#). The similarity rule may need domain-specific co-occurrence when Q-matrices are single-skill; GT cross-validation generalises wherever expert DAGs exist; DDR applies to any directed curriculum graph consumed by KT or pretraining pipelines.

Implications for graph-enhanced KT.

The protocol shows that KC graph construction should be treated as a first-class experimental component rather than an unexamined preprocessing step. Fold-specific edge inference, provenance logging, and DAG validation make it possible to identify leakage risks and graph construction choices that would otherwise remain hidden. The DDR results further show that generic graph augmentations can substantially disturb prerequisite structure, especially when they operate at the node level, whereas a prerequisite-preserving operator that protects the transitive-reduction backbone is measurably less disruptive at the same perturbation budget. The exploratory DGEKT-only downstream stress test in Section 4.7 adds an important qualification: on *this* backbone and on ASSISTments/XES3G5M only, large structural disruption did not produce a measurable AUC drop in our retraining sweep. We do not extrapolate that null predictive sensitivity to GIKT, GKT, SKT, DyGKT, or other graph consumers. Future augmentation papers should report both structural damage (DDR) and backbone-specific downstream retraining evidence.

Cold-start reporting.

The cold-start diagnostic shows why KT studies should not rely only on aggregate AUC. Rare knowledge components may behave differently from frequent knowledge components, and apparent gains or high scores on small strata may reflect class imbalance rather than genuine modelling improvement. Reporting per-stratum metrics gives reviewers a clearer view of whether a method is robust across the KC frequency distribution. In our results, graph-aware models show their narrowest real-data advantage in the very-cold stratum, while synthetic hot-stratum gains reflect clean concept structure rather than cold-start relief. Figure 3 links this diagnostic to concrete graph topology on the densest benchmark; sparser E_{pre} on ASSISTments and XES3G5M (Supplementary Figure S1) still exhibit cold strata.

Behavioural versus expert prerequisite curation.

The ground-truth cross-validation on Junyi (Section 4.9) shows that train-only behavioural inference and expert prerequisite annotation are correlated at the node

level but structurally distinct at the edge level. Two implications follow. First, behavioural inference cannot be treated as a low-cost approximation of expert curation, and papers that report graph-augmented KT results on behaviourally-inferred graphs should not claim those results transfer directly to settings with expert prerequisite graphs (or vice versa). Second, the two signals are complementary: in the most informative setup, a graph-augmented KT model would consume both an inferred behavioural graph and an expert prerequisite graph as separate relations, rather than treating either as a substitute for the other. We do not test this combined setting in the present paper, but the cross-validation provides the quantitative basis for proposing it.

Implications for graph-KT benchmarking.

The empirical observation in Section 5.1 has direct implications for how the graph-KT subfield reports gains. If leakage-controlled evaluation changes the relative ranking of graph-augmented and sequence-only KT—and our three-fold XES3G5M (primary) and ASSISTments diagnostics show it can on non-saturated corpora—then benchmark tables that do not control for graph- construction provenance are not directly comparable across papers. We do not advocate dismissing reported gains in the graph-KT literature; we advocate *disaggregating* them. A reported AUC improvement of Δ on a benchmark should ideally be decomposed into three components: the part that survives leakage control, the part that depends on graph-construction choices (relation type, threshold, edge cap, encoder), and the part that depends on backbone strength. The protocol in this paper supplies the first component; the cold-start diagnostic supplies a partial view of the second; the third remains an open methodological question.

A practical recommendation follows. When introducing a new graph-augmented KT model, papers should report at minimum: (a) the fold-specific provenance of every retained graph edge; (b) the cold-start stratum on which the gain is most concentrated; (c) the sequence-only baseline matched in compute and hyperparameter budget; (d) when an expert prerequisite annotation is available, the ground-truth cross-validation of the consumed graph against that annotation, using the edge / direction / reachability / node-Jaccard metric family used in Section 4.9; and (e) whether accuracy is actually sensitive to structure-preserving versus structure-damaging graph perturbations. Without these reports, an aggregate-AUC improvement is difficult to attribute. The protocol presented here makes these reports tractable.

5.3 Threats to Validity and Metric Interpretation

Three artefacts invite misreading: the near-null full-log ablation (Table 7), aggressive cycle pruning (Section 4.2), and scalar leakage metrics that are near-degenerate under learner-disjoint splits (Table 3).

The ablation null shift does *not* negate leakage control: graph statistics must not pool held-out learners regardless of backbone sensitivity. Under our filters, pooling rarely changes surviving transitions on public benchmarks; synthetic logs remain more sensitive. We therefore tier audit outputs (Section 3.1): **primary**—provenance logs, $\text{ECR}_f^{\text{flag}}=0$, and GT cross-validation when available; **empirical bounds**—full-log

ablation Δ and cold-start strata; **sanity/stress**— $\text{ECR}_f^{\text{overlap}}, |\rho_f|$, and TBM_f . Near-unity overlap and $\text{ECR}_f^{\text{flag}}=0$ are expected under honest learner-disjoint construction and do not replace provenance or GT audits.

Cycle pruning resolves conflicts by dropping the lowest-weight edge per detected cycle (Johnson enumeration, capped at 100 witnesses per round); pruned mass reflects weak or bidirectional transitions rather than arbitrary deletion.

TBM_f is a within-train temporal stress statistic at a 70% chronological boundary inside each learner’s train fold—not cross-split leakage ($\text{ECR}_f^{\text{flag}}=0$). Dense multi-skill co-assessment on XES3G5M and toy-graph structure on Synthetic C2 explain the largest values.

The audit also caught a one-step label misalignment in an early GIKT head (`src/models/gikt.py`), flagged by inconsistent fold AUCs and corrected before reporting Table 6.

Limitations.

Three-fold CV limits inferential power of standard fold-level tests ($p_{\min}=0.25$ for Wilcoxon; fragile paired t -tests on $n=3$ folds). Model-comparison claims in Section 5.1 therefore rely on fold-level ΔAUC magnitudes that are an order larger than fold-to-fold variation (Table 6); Supplementary Table S16 summarises the primary XES3G5M pairs with descriptive paired- t intervals, and Supplementary Table S11 reports paired tests for transparency only. Learner-level bootstrap intervals require full per-position prediction exports and are left to the released harness. ASSISTments yields no E_{sim} under item-overlap Jaccard; GT cross-validation is Junyi-only at $\theta=0.5$; downstream DDR retraining covers DGEKT only; calibration analysis is deferred. We do not claim other architectures would show the same null full-log shift.

6 Conclusion

We presented a leakage-controlled audit protocol for constructing and evaluating concept graphs in graph-enhanced KT. The contribution targets *Applied Intelligence* as a deployment and reproducibility layer: a practitioner checklist, provenance artefacts, tiered leakage diagnostics, DDR augmentation metrics, ground-truth cross-validation where expert graphs exist, and cold-start KC stratification. On XES3G5M (primary), ASSISTments (secondary), and Junyi (saturated sanity), the audit produces acyclic train-only prerequisite graphs after transparent cycle pruning. The DDR sweep separates augmentation families into consistent structural regimes across all three public corpora. The prerequisite-preserving operator is strictly less disruptive than edge dropping at a matched budget, confirming that DDR rewards structure-aware augmentation design. An exploratory DGEKT-only downstream stress test on ASSISTments and XES3G5M shows that this structural ordering does not automatically imply predictive sensitivity for *that* backbone ($r=0.13\text{--}0.15$; mean AUC drops near zero).

On XES3G5M, leakage-controlled auditing changes deployable backbone ranking (GKT trails *simpleKT* by ≈ 0.041 mean AUC; GIKT remains competitive). Ground-truth cross-validation on Junyi shows high node overlap but weak edge agreement.

Cold-start strata reveal a narrow graph-aware advantage only in very-cold knowledge components on real data. Practitioners can apply the five-item checklist in Section 1 before reporting new graph-augmented KT results. Reproduction artefacts are at https://github.com/tuanymc/p0_project.git.

Acknowledgements. The authors thank the maintainers of the Junyi Academy, ASSISTments, and XES3G5M datasets and of the pyKT benchmark library, whose public releases made this study possible.

Declarations

Funding. The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Competing interests. The authors have no competing interests to declare that are relevant to the content of this article.

Ethics approval. This study uses only publicly available, de-identified knowledge-tracing benchmark datasets under their respective licences. No human-subjects experiments were conducted and no ethics approval was required. No claims about real-world learning outcomes are made.

Consent to participate / Consent for publication. Not applicable; the study involves no identifiable individuals.

Data availability. All datasets analysed in this study are publicly available: Junyi Academy (PSLC DataShop, <https://pslcdatashop.web.cmu.edu/DatasetInfo?datasetId=1198>), ASSISTments 2012, and XES3G5M. Preprocessing scripts, fold-specific graph exports, audit logs, and all derived tables and figures are provided in the companion repository.

Code availability. Source code, configurations, pipeline scripts, and result artefacts are available at https://github.com/tuanymc/p0_project.git. Supplementary tables and figures are in [paper/supplementary.pdf](#).

Supplementary material. The supplementary PDF ([paper/supplementary.pdf](#)) is indexed at its opening page. Tables S1–S5: per-dataset baseline diagnostics; S6–S10: train-only versus full-log ablation; S11: paired significance tests on public benchmarks; S12–S13: cold-start strata; S14: ground-truth metrics including node Jaccard. Section S3: sequence-autocorrelation statistics; Sections S4–S5: augmentation-operator definitions and ground-truth disagreement examples. Figures S1–S2: multi-benchmark KT-graph and DDR schematics; Figure S3: DDR downstream scatter plot.

Author contributions. D. M. Tuan conceived the protocol, implemented the leakage-controlled pipeline, ran the experiments, and drafted the manuscript. N. T. Duong and N. K. Trinh contributed to the experimental implementation, data pre-processing, and result analysis. N. Q. Khanh contributed to manuscript revision and technical review. N. V. Hau and L. H. Son supervised the work, advised on the experimental design, and revised the manuscript. All authors read and approved the final manuscript.

References

- Abdelrahman G, Wang Q, Nunes BP (2023) Knowledge tracing: A survey. *ACM Computing Surveys* 55(11):1–37. <https://doi.org/10.1145/3569576>
- Badran Y, Preisach C (2024) Addressing label leakage in knowledge tracing models. *arXiv preprint arXiv:240315304* <https://doi.org/10.48550/arXiv.2403.15304>
- Bai Y, Liu Z, Guo T, et al (2025) Prerequisite relation learning: A survey and outlook. *ACM Computing Surveys* 57. <https://doi.org/10.1145/3733593>
- Chang HS, Hsu HJ, Chen KT (2015) Modeling exercise relationships in e-learning: A unified approach. In: *Proceedings of the 8th International Conference on Educational Data Mining (EDM)*, pp 532–535, URL <http://www.educationaldatamining.org/EDM2015/proceedings/short532-535.pdf>
- Cheng K, Peng L, Wang P, et al (2024) DyGKT: Dynamic graph learning for knowledge tracing. In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pp 377–388, <https://doi.org/10.1145/3637528.3671773>
- Choi Y, Lee Y, Shin D, et al (2020) EdNet: A large-scale hierarchical dataset in education. In: *International Conference on Artificial Intelligence in Education (AIED)*, pp 69–73, https://doi.org/10.1007/978-3-030-52240-7_13
- Corbett AT, Anderson JR (1994) Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction* 4(4):253–278. <https://doi.org/10.1007/BF01099821>
- Cui C, Yao Y, Zhang C, et al (2024) DGEKT: A dual graph ensemble learning method for knowledge tracing. *ACM Transactions on Information Systems* 42(3):1–24. <https://doi.org/10.1145/3638350>
- Ghosh A, Heffernan NT, Lan AS (2020) Context-aware attentive knowledge tracing. In: *Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pp 2330–2339, <https://doi.org/10.1145/3394486.3403282>
- Hamilton WL, Ying R, Leskovec J (2017) Inductive representation learning on large graphs. In: *Advances in Neural Information Processing Systems (NeurIPS)*, pp 1024–1034, <https://doi.org/10.48550/arXiv.1706.02216>
- Hassani K, Khasahmadi AH (2020) Contrastive multi-view representation learning on graphs. In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp 4116–4126, <https://doi.org/10.48550/arXiv.2006.05582>
- Heffernan NT, Heffernan CL (2014) The ASSISTments ecosystem: Building a platform that brings scientists and teachers together for minimally invasive research on human learning and teaching. *International Journal of Artificial Intelligence in Education*

- 24(4):470–497. <https://doi.org/10.1007/s40593-014-0024-x>
- Johnson DB (1975) Finding all the elementary circuits of a directed graph. *SIAM Journal on Computing* 4(1):77–84. <https://doi.org/10.1137/0204007>
- Junyi Academy (2015) Junyi academy online learning activity dataset. PSLC DataShop, URL <https://pslcdatashop.web.cmu.edu/DatasetInfo?datasetId=1198>, accessed 2026-06-08
- Kapoor S, Narayanan A (2023) Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* 4(9). <https://doi.org/10.1016/j.patter.2023.100804>
- Kaufman S, Rosset S, Perlich C, et al (2012) Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data* 6(4):1–21. <https://doi.org/10.1145/2382577.2382579>
- Kipf TN, Welling M (2017) Semi-supervised classification with graph convolutional networks. In: International Conference on Learning Representations (ICLR), <https://doi.org/10.48550/arXiv.1609.02907>
- Lee W, Chun J, Lee Y, et al (2022) Contrastive learning for knowledge tracing. In: Proceedings of the ACM Web Conference (WWW), pp 2330–2338, <https://doi.org/10.1145/3485447.3512105>
- Liu Z, Liu Q, Chen J, et al (2022) pyKT: A Python library to benchmark deep learning based knowledge tracing models. In: Advances in Neural Information Processing Systems (NeurIPS), pp 13248–13261, <https://doi.org/10.52202/068431-1347>
- Liu Z, Liu Q, Chen J, et al (2023a) simpleKT: A simple but tough-to-beat baseline for knowledge tracing. In: International Conference on Learning Representations (ICLR), <https://doi.org/10.48550/arXiv.2302.06881>
- Liu Z, Liu Q, Guo T, et al (2023b) XES3G5M: A knowledge tracing benchmark dataset with auxiliary information. In: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, pp 40643–40662, <https://doi.org/10.52202/075280-1429>
- Nakagawa H, Iwasawa Y, Matsuo Y (2019) Graph-based knowledge tracing: Modeling student proficiency using graph neural network. In: IEEE/WIC/ACM International Conference on Web Intelligence (WI), pp 156–163, <https://doi.org/10.1145/3350546.3352513>
- Pan L, Li C, Li J, et al (2017) Prerequisite relation learning for concepts in MOOCs. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL), pp 1447–1456, <https://doi.org/10.18653/v1/P17-1133>

- Pandey S, Karypis G (2019) A self-attentive model for knowledge tracing. In: Proceedings of the 12th International Conference on Educational Data Mining (EDM), pp 384–389, URL https://drive.google.com/file/d/18d_X6AXkPMhiHFQ2POarstVbX_7oMdFM
- Piech C, Bassen J, Huang J, et al (2015) Deep knowledge tracing. In: Advances in Neural Information Processing Systems (NeurIPS), pp 505–513, <https://doi.org/10.48550/arXiv.1506.05908>
- Schein AI, Popescul A, Ungar LH, et al (2002) Methods and metrics for cold-start recommendations. In: Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, pp 253–260, <https://doi.org/10.1145/564376.564421>
- Shen S, Liu Q, Chen E, et al (2020) Convolutional knowledge tracing: Modeling individualization in student learning process. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp 1857–1860, <https://doi.org/10.1145/3397271.3401288>
- Thakoor S, Tallec C, Azar MG, et al (2022) Large-scale representation learning on graphs via bootstrapping. In: International Conference on Learning Representations (ICLR), <https://doi.org/10.48550/arXiv.2102.06514>
- Tong H, Wang Z, Liu Q, et al (2022) HGKT: Introducing problem schema with hierarchical exercise graph for knowledge tracing. In: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp 405–415, <https://doi.org/10.1145/3477495.3532004>
- Tong S, Liu Q, Huang W, et al (2020) Structure-based knowledge tracing: An influence propagation view. In: IEEE International Conference on Data Mining (ICDM), pp 541–550, <https://doi.org/10.1109/ICDM50108.2020.00063>
- Veličković P, Fedus W, Hamilton WL, et al (2019) Deep graph infomax. In: International Conference on Learning Representations (ICLR), <https://doi.org/10.48550/arXiv.1809.10341>
- Xia J, Wu L, Chen J, et al (2022) SimGRACE: A simple framework for graph contrastive learning without data augmentation. In: Proceedings of the ACM Web Conference (WWW), pp 1070–1079, <https://doi.org/10.1145/3485447.3512156>
- Yang Y, Shen J, Qu Y, et al (2021) GIKT: A graph-based interaction model for knowledge tracing. In: Machine Learning and Knowledge Discovery in Databases (ECML-PKDD 2020), pp 299–315, https://doi.org/10.1007/978-3-030-67658-2_18
- You Y, Chen T, Sui Y, et al (2020) Graph contrastive learning with augmentations. In: Advances in Neural Information Processing Systems (NeurIPS), pp 5812–5823, <https://doi.org/10.48550/arXiv.2010.13902>

- Zhang J, Shi X, King I, et al (2017) Dynamic key-value memory networks for knowledge tracing. In: Proceedings of the 26th International Conference on World Wide Web (WWW), pp 765–774, <https://doi.org/10.1145/3038912.3052580>
- Zhu Y, Xu Y, Yu F, et al (2021) Graph contrastive learning with adaptive augmentation. In: Proceedings of the Web Conference (WWW), pp 2069–2080, <https://doi.org/10.1145/3442381.3449802>